



Proyecto ejemplo: entorno virtual + Gemini

Proyecto para practicar los conceptos de **entorno virtual**, **requirements.txt**, **variables de entorno** (**.env**) y una **primera llamada** a la API de Gemini desde Python.

Qué vas a aprender

- Crear y activar un entorno virtual (**.venv**) con el módulo **venv**.
- Documentar e instalar dependencias con **requirements.txt**.
- Guardar secretos en **.env** (sin subirlos a Git) y cargarlos con **python-dotenv**.
- Ejecutar un script o notebook que llama a Gemini.

Tecnologías

Tecnología	Uso en este proyecto
Python 3.10+	Lenguaje del proyecto
venv	Entorno virtual aislado (carpeta .venv)
pip	Instalar paquetes dentro del .venv
requirements.txt	Lista de dependencias reproducibles
python-dotenv	Leer variables desde el fichero .env
google-genai	SDK oficial de la Gemini API
.gitignore	No versionar .venv , .env ni __pycache__

API key: créala en [Google AI Studio](#) (gratuita con límites de uso).

Estructura del proyecto

```
ejemplo_proyecto_entornos_virtuales/  
├── README.md           ← este documento  
├── requirements.txt     ← dependencias del proyecto  
├── .gitignore          ← qué no subir a Git  
├── .env.example        ← plantilla (sí va al repo)  
├── .env                ← tu clave real (lo creas tú; no en Git)  
├── gemini_hola_mundo.py ← script de prueba (terminal)  
├── gemini_hola_mundo.ipynb ← mismo flujo en celdas (IDE / Jupyter)  
└── .venv/              ← entorno virtual (lo creas tú; no en Git)
```

El script y el notebook hacen lo mismo: cargar la API key, crear el cliente y pedir un resumen corto al modelo `gemini-3-flash-preview`.

Requisitos previos

- Python 3.10 o superior instalado (`python --version` o `python3 --version`).
- Terminal (PowerShell, cmd, Git Bash, macOS Terminal, etc.).
- Editor opcional: VS Code, Cursor o Jupyter para el `.ipynb`.
- Clave `GEMINI_API_KEY` de Google AI Studio.

Tutorial paso a paso (terminal)

Abre la terminal en **esta carpeta** (`ejemplo_proyecto_entornos_virtuales`) y sigue los pasos.

Paso 1 — Crear el fichero `.env` con tu API key

Copia la plantilla y edita la clave:

```
# Windows (PowerShell)  
copy .env.example .env  
  
# macOS / Linux / Git Bash  
cp .env.example .env
```

Abre `.env` y sustituye el valor de ejemplo por tu clave real:

```
GEMINI_API_KEY=AIza...tu_clave_real...
```

Importante: no subas `.env` a Git. Ya está listado en `.gitignore`.

Paso 2 — Crear el entorno virtual

```
python -m venv .venv
```

En macOS/Linux, si el comando es `python3`:

```
python3 -m venv .venv
```

Paso 3 — Activar el entorno virtual

Tras activar, el prompt suele mostrar `(.venv)` al inicio.

Sistema	Comando
Windows (PowerShell)	<code>.\.venv\Scripts\Activate.ps1</code>
Windows (cmd)	<code>.venv\Scripts\activate.bat</code>
Windows (Git Bash)	<code>source .venv/Scripts/activate</code>
macOS / Linux	<code>source .venv/bin/activate</code>

Comprobar que usas el Python del proyecto:

```
python -c "import sys; print(sys.executable)"
```

La ruta debe incluir `ejemplo_proyecto_entornos_virtuales` y `.venv`.

Desactivar cuando termines:

```
deactivate
```

PowerShell: error al activar

Si aparece *"la ejecución de scripts está deshabilitada"*:

- Usa **cmd** o **Git Bash**, o
- En PowerShell (solo esta sesión): `Set-ExecutionPolicy Unrestricted -Scope Process`
- Recomendado (usuario): `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`

Paso 4 — Instalar dependencias desde `requirements.txt`

Con `(.venv)` activo:

```
python -m pip install -U pip
```

```
python -m pip install -r requirements.txt
```

Contenido de `requirements.txt`:

```
google-genai>=1.0.0  
python-dotenv>=1.0.0
```

Así instalas el SDK de Gemini y `python-dotenv` **solo** en este entorno, sin ensuciar el Python global.

Paso 5 — Ejecutar el proyecto de prueba

```
python gemini_hola_mundo.py
```

Qué debería pasar:

1. `load_dotenv()` lee `GEMINI_API_KEY` desde `.env`.
2. Si no hay clave, te la pide con `getpass` (entrada oculta).
3. Imprime `GEMINI_API_KEY configurada: sí`.
4. Muestra un texto generado por Gemini (resumen de la IA generativa).

Si ves un error de módulo (`google` o `dotenv`), vuelve al Paso 3 y 4: el `.venv` no está activo o no instalaste `requirements.txt`.

Usar el notebook (`gemini_hola_mundo.ipynb`)

1. Abre `gemini_hola_mundo.ipynb` en VS Code, Cursor o Jupyter.
2. **Selecciona el kernel** del Python de esta carpeta:
 - Windows: `...\ejemplo_proyecto_entornos_virtuales\.venv\Scripts\python.exe`
 - macOS/Linux: `.../ejemplo_proyecto_entornos_virtuales/.venv/bin/python`
3. Ejecuta las celdas en orden:
 - **Celda 0:** instalación (`%pip install -U google-genai python-dotenv`) si hace falta.
 - **Celda 1:** `load_dotenv()` y comprobación de la API key.
 - **Celda 2:** primera llamada a Gemini.

En una celda puedes comprobar el intérprete:

```
import sys  
print(sys.executable)
```

Kernel en Jupyter clásico (opcional), con `.venv` activo:

```
python -m pip install ipykernel
python -m ipykernel install --user --name=entornos-gemini --display-name="Python
(entornos gemini)"
```

Cómo funciona el código (resumen)

- 1. `load_dotenv()` — Lee `.env` y rellena `os.environ` (por ejemplo `GEMINI_API_KEY`).
- 2. **Respaldo con `getpass`** — Si no hay clave en `.env` ni en el sistema, la pide en consola (solo esa sesión).
- 3. `genai.Client()` — El SDK usa `GEMINI_API_KEY` del entorno automáticamente.
- 4. `generate_content` — Envía un prompt al modelo y muestra `response.text`.

No pegues la API key en el código ni en celdas que vayas a subir a Git.

requirements.txt en la práctica

Acción	Comando
Instalar todo lo del proyecto	<code>pip install -r requirements.txt</code>
Añadir un paquete nuevo	<code>pip install nombre_paquete</code>
Actualizar el fichero (con <code>.venv</code> activo)	<code>pip freeze > requirements.txt</code>

En proyectos reales conviene fijar versiones cuando el equipo crece (`paquete==1.2.3` o `paquete>=1.0.0` como aquí).

Variables de entorno y Git

Fichero	¿Subir a Git?	Contenido
<code>.env.example</code>	Sí	Plantilla sin clave real
<code>.env</code>	No	Tu <code>GEMINI_API_KEY</code> real
<code>.venv/</code>	No	Entorno virtual de tu máquina
<code>requirements.txt</code>	Sí	Dependencias
Código <code>.py</code> / <code>.ipynb</code>	Sí	Lógica del proyecto

Alternativa sin `.env`: exportar la variable antes de ejecutar:

```
# PowerShell
$env:GEMINI_API_KEY = "tu_clave"

# Git Bash / macOS / Linux
export GEMINI_API_KEY="tu_clave"
```

Resumen de comandos

```
cd ruta/a/ejemplo_proyecto_entornos_virtuales

cp .env.example .env
# editar .env con tu GEMINI_API_KEY

python -m venv .venv
# activar .venv (según tu sistema)

python -m pip install -U pip
python -m pip install -r requirements.txt

python gemini_hola_mundo.py
```

Checklist

- ☐ Estoy en la carpeta `ejemplo_proyecto_entornos_virtuales`.
- ☐ Existe `.env` con mi `GEMINI_API_KEY` (creado desde `.env.example`).
- ☐ Existe `.venv` y está **activado** (o el kernel del notebook apunta a ese `.venv`).
- ☐ `pip install -r requirements.txt` terminó sin errores.
- ☐ `python gemini_hola_mundo.py` o el notebook muestra texto del modelo.